

DATA ENGINEERING CAPSTONE PROJECT

Problem Statement #03

Ride-Hailing Surge Analytics

Project Proposal: Problem Statement, Scope, Architecture, Tools and Implementation Approach

Submitted by

Bharadwaj Boyapati

Khuraijam Richard

Moirangthem Erudite

Implementation language: Python (PySpark)
September 2026

Table of Contents

1. Executive Overview.....	3
2. Problem Statement.....	3
2.1 Business Context.....	3
2.2 Core Problem Statement.....	3
2.3 Why This Matters.....	3
3. Objectives.....	3
4. Key Performance Indicators and SLAs.....	4
5. Project Scope.....	4
5.1 In Scope.....	4
5.2 Out of Scope.....	5
5.3 Assumptions and Constraints.....	5
6. Technology Stack.....	5
6.1 Tools by Layer (as specified).....	5
6.2 Python Implementation Mapping.....	6
7. System Architecture.....	7
7.1 Layer 1 — Ingestion.....	7
7.2 Layer 2 — Speed & Batch Engine.....	7
7.3 Layer 3 — Storage & Lakehouse.....	7
7.4 Layer 4 — Orchestration & AI.....	8
8. Data Sources.....	8
9. Detailed Module Design and Process.....	8
9.1 Ingestion and Streaming.....	8
9.2 Processing and Cleansing.....	9
9.3 Surge Multiplier Logic.....	10
9.4 Storage and Data Modelling.....	11
9.5 Cloud and Lakehouse Stack.....	12
9.6 Orchestration and Workflow (Airflow).....	13
9.7 Analytics and AI Integration.....	13
10. Live Stream and Replay Hybrid Design.....	14
11. Reliability and Failure Drills.....	14
12. Measurable Outcomes and Acceptance.....	15
13. Implementation Plan.....	15
13.1 Phases.....	15
13.2 Proposed Work Split.....	16
13.3 Repository Structure.....	16
14. Deliverables.....	16
15. Risks and Mitigations.....	17
16. Conclusion.....	17

1. Executive Overview

This document describes the plan for our capstone project, **Ride-Hailing Surge Analytics**. It covers the problem we are solving, the scope, the end-to-end architecture, the tools we will use, and the step-by-step process we will follow to build, test and demonstrate the system.

The project builds a near-real-time data platform for an urban cab aggregator. The platform ingests trip requests, driver GPS pings and live weather, computes the demand-to-supply ratio for every city zone every 60 seconds, and recommends a surge multiplier for each zone. Alongside the real-time path, a batch and lakehouse path stores history, feeds dashboards, trains a 30-minute demand forecast model and powers a GenAI operations assistant.

All tools listed in the problem statement are retained exactly as specified. The only change is the programming language: every custom component (Kafka producer, Spark jobs, Airflow DAGs, ML and GenAI code) will be written in **Python** instead of Scala. Section 6.2 maps each specified item to its Python implementation.

2. Problem Statement

2.1 Business Context

An urban cab aggregator experiences driver shortages and high rider wait times during peak hours, because demand and supply are not balanced dynamically across city zones. When a zone has many requests and few available drivers, riders wait longer or cancel, while drivers in quieter zones sit idle. Without timely, zone-level visibility, pricing and driver positioning cannot react fast enough.

2.2 Core Problem Statement

Build a near-real-time streaming pipeline ingesting trip requests, driver GPS pings, and weather data to compute zone-level demand/supply ratios every 60 seconds and recommend optimal surge multipliers.

2.3 Why This Matters

- **Riders:** shorter waits, because surge pricing draws drivers into under-supplied zones.
- **Drivers:** higher utilisation and earnings, because they are guided to zones with demand.
- **Platform:** more completed trips, fewer cancellations, and lower cloud analytics cost through efficient storage.
- **Operations team:** a live view of every zone and a natural-language assistant to ask questions about the business.

3. Objectives

1. Ingest historical data (trips, drivers, riders, city zones) from MySQL in batch, and live events (trip requests, driver GPS pings, weather) through Kafka.
2. Compute zone-level demand, supply and demand/supply ratio on a 5-minute sliding window that moves every 1 minute, and publish a surge multiplier per zone every 60 seconds.
3. Store the current surge multipliers in HBase for low-latency lookup, and store history in a Hive star schema and a Databricks Delta Medallion lakehouse.

4. Store analytical data as partitioned Parquet on S3, catalogued in Glue, so Athena queries scan at least 70% less data than the raw format.
5. Orchestrate batch work with Airflow: an hourly zone aggregation DAG and a daily driver incentive payout DAG with automated email reports.
6. Build a demand forecasting model (Linear Regression / Random Forest) that predicts 30-minute forward demand per zone with $R^2 \geq 0.70$.
7. Deliver Power BI dashboards and a GenAI Ops assistant that answers questions by querying Gold tables through tool calls.
8. Prove reliability through three failure drills: broker kill, Spark restart and record-count reconciliation.

4. Key Performance Indicators and SLAs

These targets are taken directly from the problem statement. Section 12 describes how each one will be measured.

Metric Type	Key Performance Indicator	Target	Business Impact
Business	Average Rider ETA	< 4.5 minutes	Improves rider satisfaction and retention
Business	Driver Utilisation Rate	$\geq 78\%$	Maximises driver earnings and platform revenue
Operational	Surge Multiplier Refresh	Every 60 seconds	Dynamically balances zone supply/demand
Technical SLA	Athena Scan Cost Reduction	$\geq 70\%$ cost drop	Optimises cloud analytics query cost
Model SLA	Demand Forecast Model	$R^2 \geq 0.70$	Accurately predicts 30-min forward demand

5. Project Scope

5.1 In Scope

- Batch ingestion of historical trips, drivers, riders and city zones from MySQL using Sqoop into the HDFS raw landing zone.
- A hybrid live + replay Kafka feed: CityBikes API (supply proxy), Open-Meteo (live weather) and NYC TLC trip records replayed as trip requests, plus a driver GPS ping simulator.
- Spark Core batch analysis of historical trips (peak hours and cancellation reasons using pair RDDs).
- Spark Structured Streaming surge engine with 5-minute windows, 1-minute slide, deduplication and a join with live weather.
- HBase table of surge multipliers keyed by `zone_id#minute`; Hive star schema partitioned by city and date.
- S3 Parquet storage with Glue Data Catalog and Athena; Databricks Medallion (Bronze/Silver/Gold) with Delta Live Tables quality expectations.
- Airflow master DAG, hourly zone aggregation DAG and daily driver incentive payout DAG with email reports.

- Power BI dashboards: live zone heat map, driver utilisation gauge and revenue/km trends.
- Demand forecasting model (Linear Regression and Random Forest) and a GenAI Ops assistant using tool calls over Gold tables.
- Failure drills (broker kill, Spark restart, reconciliation) and a final demo.

5.2 Out of Scope

- A rider or driver mobile app, real payments, or real dispatch/matching of drivers to riders.
- Production-grade security hardening (SSO, network isolation, secrets vaults beyond environment variables).
- Multi-region deployment and autoscaling of the Kafka and Hadoop clusters.
- Real personal data: all driver and rider identities are synthetic.

5.3 Assumptions and Constraints

- **City and zones:** New York City is used as the reference city because the NYC TLC trip data, taxi zone map (263 zones) and Citi Bike stations all cover it. Zone = TLC **LocationID**.
- **Supply signal:** Driver supply comes from a Python driver GPS simulator (synthetic fleet), with CityBikes dock availability as the supply proxy specified in the problem statement.
- **Driver identities:** Public TLC data has no driver IDs, so drivers and riders are generated with the Faker library and assigned to historical trips.
- **Infrastructure:** Open-source components run locally in Docker Compose; S3, Glue and Athena use the AWS Free Tier; Databricks uses the Databricks Free Edition. A laptop with at least 16 GB RAM is needed to run the full stack.
- **Data volume:** One or two months of NYC high-volume for-hire vehicle (HVFHV) records are enough to train the model and demonstrate the partitioning benefit.

6. Technology Stack

6.1 Tools by Layer (as specified)

Layer	Tool (as in problem statement)	Role in this project
Source	MySQL	Operational database holding trips, drivers, riders and city_zones
Ingestion	Apache Sqoop	Batch import of MySQL tables into HDFS
Ingestion	Apache Kafka (3 brokers)	Live event bus for trip requests, GPS pings and weather
Ingestion	HDFS	Raw landing zone for batch imports and archived stream data
Processing	Spark Core	Batch analysis of historical trips with pair RDDs
Processing	Spark Structured Streaming	Windowed demand/supply aggregation and surge calculation
Storage	Apache HBase	Low-latency key-value store for current surge multipliers
Storage	Apache Hive	Star schema data warehouse partitioned by city and date

Layer	Tool (as in problem statement)	Role in this project
Cloud	Amazon S3 + AWS Glue Data Catalog + Amazon Athena	Parquet data lake, metadata catalog and serverless SQL
Lakehouse	Databricks + Delta Lake + Delta Live Tables	Bronze/Silver/Gold Medallion with quality expectations
Orchestration	Apache Airflow	Master, hourly zone aggregation and daily payout DAGs
BI	Power BI / Tableau	Heat map, utilisation gauge, revenue/km trends
ML	Linear Regression / Random Forest	30-minute forward demand forecast per zone
GenAI	LLM-based Ops assistant with tool calls	Natural-language questions answered from Gold tables
External data	CityBikes API, Open-Meteo API, NYC TLC trip records	Live supply proxy, live weather, replayed trip requests

6.2 Python Implementation Mapping

The problem statement names Scala for the custom producer. We will implement every custom component in Python while keeping the same tools and behaviour:

Specified item	Python implementation	Library / API
Custom Scala Kafka producer	Python producers for trip requests, GPS pings and weather	<code>confluent-kafka</code> , <code>requests</code> , <code>pandas/pyarrow</code>
Spark Core pair RDDs	PySpark RDD API: <code>map</code> → <code>(key, value)</code> , <code>reduceByKey</code> , <code>sortBy</code>	<code>pyspark</code> (RDD)
<code>reduceByKeyAndWindow</code> (5-min window, 1-min slide)	Structured Streaming <code>groupBy(window(event_time, "5 minutes", "1 minute"), zone_id)</code>	<code>pyspark.sql.functions.window</code>
HBase writes	Writes from Spark <code>foreachBatch</code> through the HBase Thrift server	<code>happybase</code>
Hive star schema	HiveQL DDL + PySpark with <code>enableHiveSupport()</code>	<code>pyspark.sql</code> , <code>HiveServer2</code>
Sqoop import	Sqoop CLI commands triggered from Airflow	Airflow <code>BashOperator</code>
S3 / Glue / Athena	Upload, crawler runs and query benchmarking	<code>boto3</code> , <code>aws wrangler</code>
Delta Live Tables	Python DLT pipeline with <code>@dlt.table</code> and <code>@dlt.expect_or_drop</code>	<code>dlt</code> (Databricks)
Airflow DAGs	Python DAG files	<code>apache-airflow</code>
Demand forecast model	Linear Regression and Random Forest regressors, tracked in MLflow	<code>scikit-learn</code> , <code>mlflow</code>
GenAI Ops assistant	LLM with tool/function calling and a chat UI	LLM SDK, <code>databricks-sql-connector</code> , <code>streamlit</code>

Note: `reduceByKeyAndWindow` is a function that belongs to Spark's older DStream API, not to Structured Streaming. PySpark Structured Streaming achieves exactly the same result, a 5-minute

window sliding every 1 minute per key, with `groupBy(window(...), key)`. We use the Structured Streaming form because it supports watermarks, exactly-once checkpointing and Kafka offsets natively, which the failure drills depend on.

7. System Architecture

The platform follows the four-layer architecture in the problem statement (Figure 03.1), with the data sources shown as an additional column on the left for clarity.

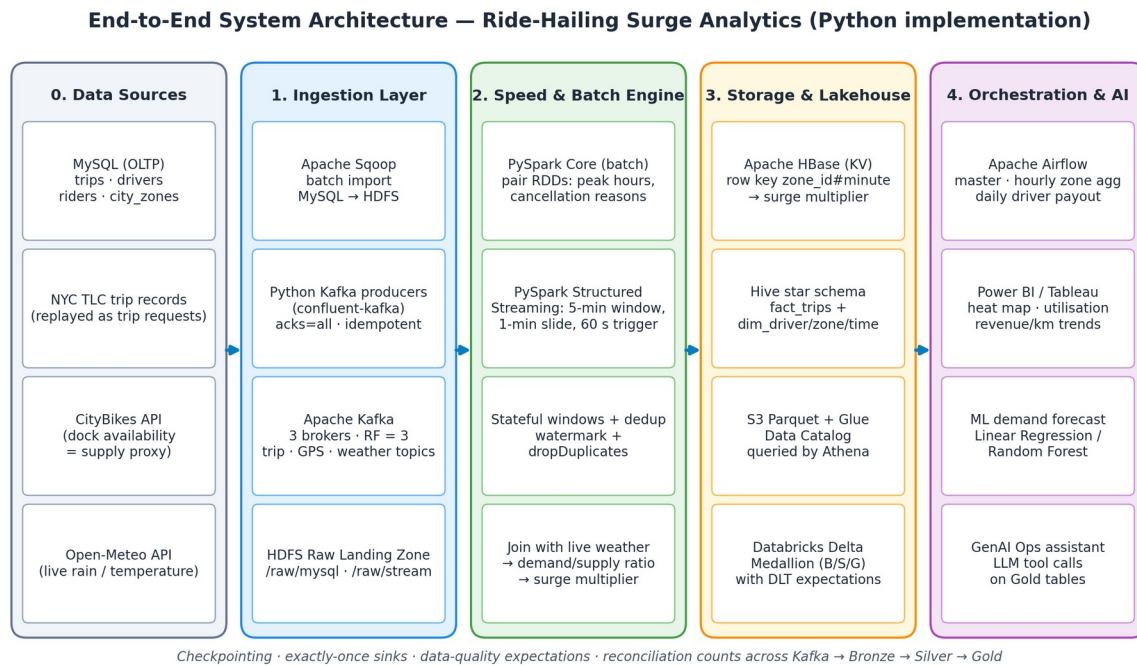


Figure 1: End-to-end architecture — data sources, ingestion, speed & batch engine, storage & lakehouse, orchestration & AI

7.1 Layer 1 — Ingestion

- **MySQL (Sqoop batch):** historical trips, drivers, riders and city zones are imported nightly into HDFS as Parquet.
- **Kafka (live API feed):** Python producers publish trip requests, driver GPS pings and weather to a 3-broker Kafka cluster.
- **HDFS raw landing zone:** immutable copy of every batch import and of the raw stream, used for replays and reconciliation.

7.2 Layer 2 — Speed & Batch Engine

- **Spark Structured Streaming** (speed layer) computes demand, supply and surge per zone every 60 seconds.
- **Spark Core cleansing job** (batch layer) cleans historical data and analyses peak hours and cancellation reasons.
- **Stateful windows & dedup** use watermarks and event IDs so late or repeated events are handled correctly.

7.3 Layer 3 — Storage & Lakehouse

- **HBase** serves the latest surge multiplier per zone with millisecond reads.

- **Databricks Delta (Bronze/Silver/Gold)** holds the curated lakehouse with quality rules.
- **Hive star schema** and **S3 Parquet + Glue + Athena** provide the warehouse and low-cost SQL analytics.

7.4 Layer 4 — Orchestration & AI

- **Airflow master DAG** schedules and chains all batch work.
- **Power BI / Tableau** dashboards visualise zones, utilisation and revenue.
- **ML models and GenAI agent** forecast demand and answer operational questions.

8. Data Sources

Source	Type	Key fields	Used for
MySQL trips	Batch (Sqoop)	trip_id, driver_id, rider_id, pickup/dropoff zone, request/pickup/dropoff time, distance, fare, status, cancel_reason	History, star schema, model training
MySQL drivers , riders	Batch (Sqoop)	id, name (synthetic), vehicle_type, joined_date, home_city, rating	Dimensions, incentive payouts
MySQL city_zones	Batch (Sqoop)	zone_id (TLC LocationID), zone_name, borough, service_zone, centroid	Zone dimension, mapping GPS to zones
NYC TLC HVFHV trip records	Replay → Kafka	request_datetime, pickup_datetime, PULocationID, DOLocationID, trip_miles, base_passenger_fare	Trip request stream (demand) and seed data for MySQL
Driver GPS simulator	Live → Kafka	driver_id, lat, lon, status (available / on_trip), ts	Supply stream and utilisation
CityBikes API (citi-bike-nyc)	Live → Kafka	station id, lat, lon, free_bikes, empty_slots, timestamp	Supply proxy per zone
Open-Meteo API	Live → Kafka	temperature_2m, precipitation, rain, weather_code	Weather adjustment to surge and forecast feature

Seed data flow: TLC Parquet files are loaded into MySQL by a Python script (**pandas** + **SQLAlchemy**), with synthetic driver and rider IDs attached. This gives Sqoop a realistic operational source to import from, as the problem statement requires.

9. Detailed Module Design and Process

9.1 Ingestion and Streaming

Batch: Sqoop import from MySQL

Each MySQL table is imported into the HDFS raw zone as Parquet. Trips are imported incrementally on **last_modified** so reruns do not duplicate data. Airflow triggers the commands.

```
sqoop import --connect jdbc:mysql://mysql:3306/ridehail \
--username etl --password-file /secrets/mysql.pwd \
--table trips --target-dir /raw/mysql/trips \
--as-parquetfile --incremental lastmodified \
--check-column last_modified --merge-key trip_id -m 4
```

Streaming: Kafka topics

Topic	Producer	Key	Partitions / RF	Rate
<code>trip_requests</code>	TLC replay producer	<code>zone_id</code>	6 / 3	Replayed at configurable speed (e.g. 1× – 60×)
<code>driver_gps_pings</code>	GPS simulator + CityBikes poller	<code>zone_id</code>	6 / 3	Every 15–30 s per driver; CityBikes every 60 s
<code>weather</code>	Open-Meteo poller	<code>city</code>	1 / 3	Every 60 s
<code>surge_updates</code>	Streaming job (output)	<code>zone_id</code>	6 / 3	Every 60 s

Producer reliability settings: `acks=all`, `enable.idempotence=true`, and topic `min.insync.replicas=2`. With 3 brokers and replication factor 3, one broker can fail with no data loss (Drill 1). Every event carries a unique `event_id` and an `event_time` for deduplication and windowing.

```
from confluent_kafka import Producer
import json

producer = Producer({
    "bootstrap.servers": "kafka1:9092,kafka2:9092,kafka3:9092",
    "acks": "all",
    "enable.idempotence": True,
})

def send_trip_request(row):
    event = {
        "event_id": row.event_id,
        "event_time": row.request_datetime.isoformat(),
        "zone_id": int(row.PULocationID),
        "dropoff_zone_id": int(row.DOLocationID),
    }
    producer.produce("trip_requests", key=str(event["zone_id"]),
                    value=json.dumps(event))
    producer.poll(0)
```

9.2 Processing and Cleansing

Spark Core batch analysis (pair RDDs)

The Spark Core job reads historical trips from HDFS, removes invalid records (null zones, negative fares, drop-off before pick-up), and produces two analyses using pair RDDs:

- **Peak hours:** (`hour_of_day`, 1) pairs reduced with `reduceByKey` to find the busiest hours per zone and day type.
- **Cancellation reasons:** (`cancel_reason`, 1) pairs reduced and sorted to rank the causes of cancellations by zone and hour.

```
trips = sc.textFile("hdfs:///raw/mysql/trips_csv").map(parse).filter(is_valid)
peak_hours = (trips.map(lambda t: (t.pickup_ts.hour, 1))
               .reduceByKey(lambda a, b: a + b)
               .sortBy(lambda kv: -kv[1]))
cancel_reasons = (trips.filter(lambda t: t.status == "CANCELLED")
                  .map(lambda t: (t.cancel_reason, 1))
                  .reduceByKey(lambda a, b: a + b))
```

Spark Structured Streaming surge engine

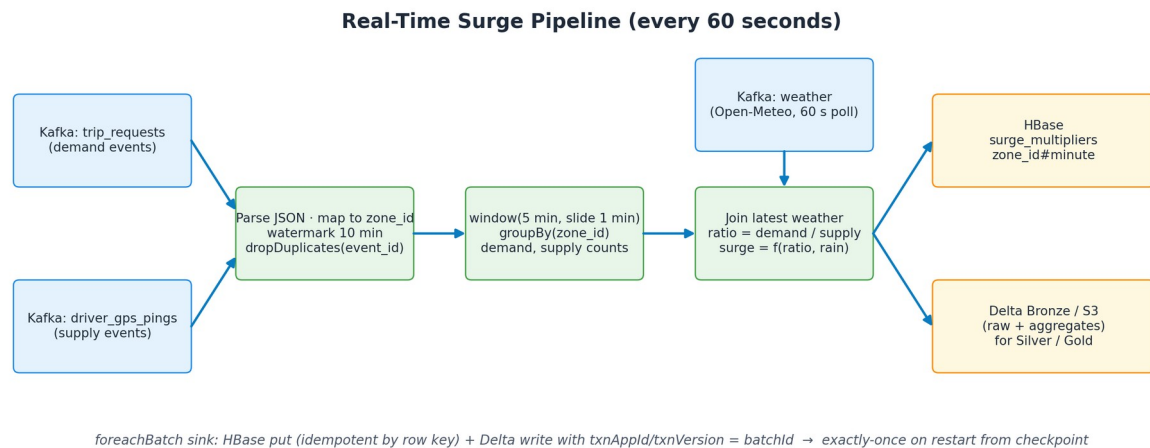


Figure 2: Real-time surge pipeline, refreshed every 60 seconds

1. Read `trip_requests` and `driver_gps_pings` from Kafka; parse JSON; map GPS coordinates to a `zone_id` (point-in-polygon on the TLC zone shapes, pre-computed for CityBikes stations).
2. Apply a 10-minute watermark on `event_time` and `dropDuplicates` on `event_id`.
3. Aggregate on a **5-minute window sliding every 1 minute** per zone: demand = count of trip requests; supply = distinct available drivers + CityBikes supply proxy.
4. Join each window with the latest weather reading for the city (rain, temperature).
5. Compute `ratio = demand / max(supply, 1)` and the surge multiplier (Section 9.3).
6. Every 60 seconds (`trigger(processingTime="60 seconds")`), write results with `foreachBatch` to HBase, the `surge_updates` topic and the Delta Bronze layer.

```
from pyspark.sql import functions as F

demand = (requests.withWatermark("event_time", "10 minutes")
          .dropDuplicates(["event_id", "event_time"])
          .groupBy(F.window("event_time", "5 minutes", "1 minute"), "zone_id")
          .agg(F.count("*").alias("demand")))

query = (zone_surge.writeStream
        .foreachBatch(write_to_hbase_and_delta)
        .option("checkpointLocation", "hdfs:///checkpoints/surge")
        .trigger(processingTime="60 seconds")
        .outputMode("update").start())
```

9.3 Surge Multiplier Logic

The first version uses a transparent rule-based policy so results can be explained and tuned. The demand forecast can later be added as an input.

Demand / supply ratio	Base multiplier
< 1.0	1.0× (no surge)
1.0 – 1.5	1.2×
1.5 – 2.0	1.5×
2.0 – 3.0	2.0×
> 3.0	2.5×

- **Weather adjustment:** +0.2× when rain $\geq$ 2.5 mm/h; +0.1× when temperature is below 5 °C or above 35 °C.
- **Cap and smoothing:** final multiplier is capped at 3.0×, and can change by at most $\pm 0.3\times$ per minute to avoid flickering prices.
- These thresholds are starting values and will be tuned against the ETA and utilisation KPIs using the replayed data.

9.4 Storage and Data Modelling

HBase — current surge multipliers

Item	Design
Table	<code>surge_multipliers</code>
Row key	<code>zone_id#minute</code> , e.g. <code>132#202609291145</code> (zone ID zero-padded to 3 digits so keys sort correctly)
Column family <code>s</code>	<code>multiplier</code> , <code>demand</code> , <code>supply</code> , <code>ratio</code> , <code>rain_mm</code> , <code>temp_c</code> , <code>updated_at</code>
Access pattern	Latest value for a zone = prefix scan on <code>zone_id#</code> in reverse; history of a zone for the last hour = range scan
Retention	TTL of 7 days; long-term history lives in Delta and Hive

Hive star schema

The warehouse uses one fact table and three dimensions, stored as Parquet and **partitioned by city and date** as specified.

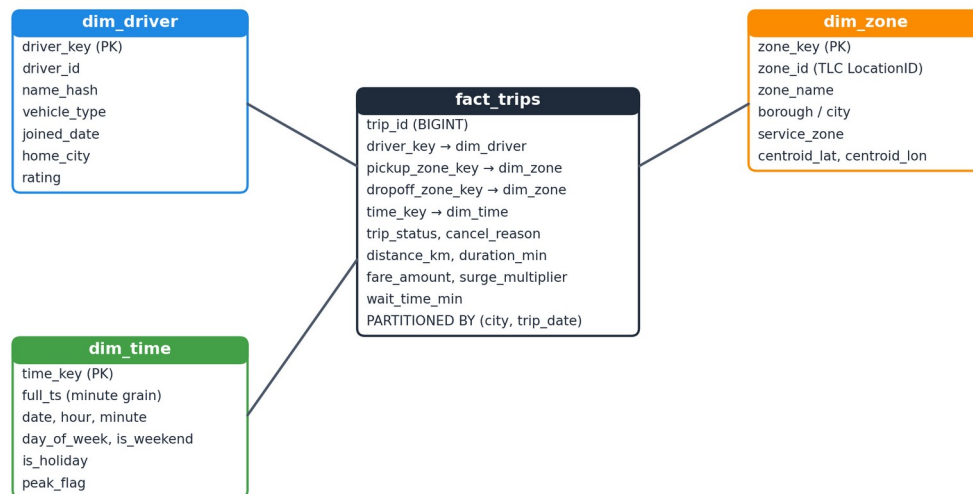
Hive Star Schema (partitioned by city, trip_date)

Figure 3: Hive star schema — fact_trips with dim_driver, dim_zone and dim_time

```

CREATE EXTERNAL TABLE fact_trips (
  trip_id BIGINT, driver_key INT, pickup_zone_key INT, dropoff_zone_key INT,
  time_key BIGINT, trip_status STRING, cancel_reason STRING,
  distance_km DOUBLE, duration_min DOUBLE, fare_amount DOUBLE,
  surge_multiplier DOUBLE, wait_time_min DOUBLE)
PARTITIONED BY (city STRING, trip_date DATE)
STORED AS PARQUET LOCATION 'hdfs:///warehouse/fact_trips';
  
```

9.5 Cloud and Lakehouse Stack

S3 Parquet + Glue Data Catalog + Athena

- Curated tables are written to S3 as Snappy-compressed Parquet under `s3://<bucket>/curated/<table>/city=<city>/trip_date=<date>/`.
- A Glue crawler (run by Airflow) registers tables and new partitions in the Glue Data Catalog.
- Athena queries the catalogued tables. Queries filter on `city` and `trip_date`, so Athena reads only the needed partitions and columns.

Databricks Medallion with Delta Live Tables

Layer	Content	Quality rules
Bronze	Raw events and batch extracts as received (Auto Loader from S3 landing)	None — full fidelity for replay and audit
Silver	Cleaned, deduplicated, typed and zone-mapped trips, pings and weather	Valid GPS coordinates; fare > 0; non-null zone_id; pickup after request
Gold	zone_minute_surge, zone_hourly_kpis, driver_daily_utilisation, revenue_per_km, demand_forecast	Aggregates reconcile with Silver counts

```
import dlt
from pyspark.sql import functions as F

@dlt.table(comment="Cleaned trip requests")
@dlt.expect_or_drop("valid_gps",
    "lat BETWEEN 40.4 AND 41.0 AND lon BETWEEN -74.3 AND -73.6")
@dlt.expect_or_drop("positive_fare", "fare_amount > 0")
def silver_trips():
    return dlt.read_stream("bronze_trips").dropDuplicates(["event_id"])
```

Rows that fail an expectation are dropped and counted in the DLT event log. These counts feed the reconciliation drill. (Databricks now also calls DLT *Lakeflow Declarative Pipelines*; the Python `dlt` API is the same.)

9.6 Orchestration and Workflow (Airflow)

DAG	Schedule	Tasks
<code>master_batch_dag</code>	Daily 00:30	Sqoop import (4 tables) → Spark Core cleansing job → load Hive partitions → sync to S3 → run Glue crawler → trigger downstream DAGs
<code>hourly_zone_aggregation</code>	Every hour	Wait for Silver data → aggregate trips, demand, supply, surge, ETA and revenue per zone-hour → write Gold <code>zone_hourly_kpis</code> → data-quality check → refresh Athena partitions
<code>daily_driver_incentive_payout</code>	Daily 01:00	Compute per-driver trips, online hours, utilisation and revenue → apply incentive rules → write Gold <code>driver_payouts</code> → build HTML/CSV report → send email (<code>EmailOperator</code>)

Illustrative incentive rule: a driver with utilisation $\geq 78\%$ and at least 12 completed trips in a day earns a bonus; drivers who accept trips in surge zones earn an extra per-trip amount. Failures in any DAG trigger an alert email with retries configured (`retries=2`).

9.7 Analytics and AI Integration

Power BI dashboards

- **Live heat map by city zone:** surge multiplier and demand/supply ratio on the NYC taxi zone map (shape map visual), with page auto-refresh through DirectQuery on the Gold tables.
- **Driver utilisation gauge:** current and daily utilisation against the 78% target.
- **Revenue/km trends:** by zone, hour and weather condition; plus average rider ETA against the 4.5-minute target.

Power BI connects to the Databricks Gold tables (Databricks connector) and to Athena (ODBC). Tableau can use the same sources.

Demand forecasting model

- **Target:** number of trip requests in a zone in the next 30 minutes.
- **Features:** zone_id, hour, day of week, weekend/holiday flag, demand lags (30 min, 60 min, same time yesterday and last week), rolling averages, rain and temperature.
- **Models:** Linear Regression as the baseline and Random Forest Regressor as the main model (scikit-learn), tracked in MLflow.

- **Validation:** time-based train/test split (no shuffling), reporting R^2 , MAE and RMSE. Acceptance: $R^2 \geq 0.70$ on the held-out period.
- **Serving:** batch scoring every 30 minutes into Gold `demand_forecast`, shown on the dashboard and available to the GenAI assistant.

GenAI Ops assistant

A Python chat assistant (Streamlit UI) lets the operations team ask questions such as "Which five zones have the highest surge right now?" or "What was driver utilisation in Midtown yesterday?". The LLM does not write free SQL; it calls a fixed set of read-only tools:

Tool	What it returns
<code>get_current_surge(zone)</code>	Latest multiplier, demand, supply and weather for a zone
<code>top_zones(metric, n)</code>	Top N zones by surge, demand or wait time
<code>driver_utilisation(date, zone)</code>	Utilisation and trips for a date and optional zone
<code>demand_forecast(zone)</code>	Next 30-minute forecast with the recent actuals
<code>kpi_summary(date)</code>	ETA, utilisation, revenue/km and surge statistics for a day

Each tool runs a parameterised query against the Gold tables using `databricks-sql-connector`, so the assistant only reads curated data and cannot modify it.

10. Live Stream and Replay Hybrid Design

To keep events flowing 24/7 and still support historical backfills, the Kafka producers combine live APIs with replayed history, as specified in the problem statement:

Feed	Mode	How it works
CityBikes API	Live	Polls the Citi Bike NYC network every 60 s; each station's available bikes are mapped to its taxi zone and published as the supply proxy
Open-Meteo	Live	Polls current rain and temperature for New York every 60 s (no API key needed)
NYC TLC trip records	Replay	Reads a month of HVFHV Parquet, re-stamps each request with the current time (keeping original gaps) and publishes it as a live trip request; speed-up factor is configurable
Driver GPS simulator	Live (synthetic)	Moves a synthetic fleet between zones, marking drivers available or on_trip as replayed trips are matched
Backfill	Replay	The same replay producer can re-publish any historical date range to rebuild Bronze/Silver/Gold after a fix

11. Reliability and Failure Drills

Drill	Procedure	Pass criterion
1. Broker kill	While producers run, stop 1 of 3 Kafka brokers (<code>docker stop kafka2</code>) for 5 minutes, then restart it	Zero data loss: produced count = consumed count; surge refresh continues

Drill	Procedure	Pass criterion
2. Spark restart	Kill the streaming job mid-batch and restart it with the same checkpoint location	Job resumes from the committed offsets; no duplicate or missing rows in HBase or Delta (exactly-once)
3. Reconciliation	Compare record counts for one time range across Kafka offsets, Bronze, Silver and Gold	Counts match after accounting for rows dropped by DLT expectations and dedup

How exactly-once is achieved: Kafka offsets are stored in the Spark checkpoint. HBase writes are idempotent because the row key `zone_id#minute` is deterministic, so a replayed batch overwrites the same cells. Delta writes inside `foreachBatch` use `txnAppId` and `txnVersion = batch_id`, so Delta skips a batch it has already committed.

A Python reconciliation script produces a report per run: Kafka count (end offset – start offset), Bronze count, Silver count + dropped (from the DLT event log) + duplicates, and Gold aggregate totals.

12. Measurable Outcomes and Acceptance

Outcome	How it will be measured	Target
Surge refresh	Timestamp gap between consecutive <code>updated_at</code> values per zone in HBase	Every 60 s
Average rider ETA	request → pickup time from trip data (and simulated dispatch in live mode), averaged per zone-hour	< 4.5 min
Driver utilisation	on-trip minutes ÷ online minutes, from GPS ping status	≥ 78%
Athena scan cost	Bytes scanned for 5 benchmark queries on raw CSV vs partitioned Parquet; reduction = 1 – Parquet ÷ CSV	≥ 70% drop
Forecast accuracy	R^2 on the held-out time period	$R^2 \geq 0.70$
Fault tolerance	Results of the three drills in Section 11	All pass

13. Implementation Plan

13.1 Phases

Week	Phase	Main activities	Output
1	Setup & data	Docker Compose stack (MySQL, Kafka ×3, Hadoop/HDFS, Hive, HBase, Spark, Airflow); AWS and Databricks accounts; download TLC data and zone map; seed MySQL	Running environment, seeded MySQL
2	Batch ingestion	Sqoop imports to HDFS; Spark Core cleansing and pair-RDD analysis	Raw zone, peak-hour and cancellation reports
3	Streaming ingestion	Python producers (TLC replay, GPS simulator, CityBikes, Open-Meteo); topics with RF=3	Continuous events in Kafka
4	Surge engine	Structured Streaming windows, dedup, weather join, surge logic, HBase + Delta sinks, checkpointing	Surge per zone every 60 s

Week	Phase	Main activities	Output
5	Warehouse & cloud	Hive star schema; S3 Parquet; Glue crawler; Athena benchmark; Databricks DLT Bronze/Silver/Gold	Star schema, lakehouse, cost report
6	Orchestration	Airflow master, hourly aggregation and daily payout DAGs with email	Scheduled pipelines and reports
7	Analytics & AI	Power BI dashboards; forecast model training and scoring; GenAI assistant	Dashboards, model ($R^2 \geq 0.70$), assistant
8	Testing & demo	Failure drills, reconciliation, KPI measurement, documentation, final presentation	Drill report, final report, demo

13.2 Proposed Work Split

Each member owns one layer end to end; all three work together on integration, testing and the demo. The team may adjust this split.

Member	Primary ownership
Bharadwaj Boyapati	Ingestion and speed layer: MySQL seeding, Sqoop, Kafka cluster and Python producers, Structured Streaming surge engine
Khurajam Richard	Storage and lakehouse: Spark Core batch job, HBase, Hive star schema, S3/Glue/Athena, Databricks DLT, reconciliation
Moirangthem Erudite	Orchestration and AI: Airflow DAGs and email reports, Power BI dashboards, demand forecast model, GenAI Ops assistant

13.3 Repository Structure

```
ride-hailing-surge-analytics/
├── docker/                # docker-compose.yml, service configs
├── data/                  # seed scripts, TLC samples, zone shapefile
├── ingestion/
│   ├── sqoop/             # import commands
│   └── producers/         # tlc_replay, gps_simulator, citybikes, weather
├── processing/
│   ├── batch/             # spark_core_trips.py (pair RDDs)
│   └── streaming/         # surge_stream.py, surge_rules.py, sinks.py
├── storage/               # hbase_schema.py, hive_ddl.sql
├── lakehouse/             # dlt_pipeline, s3_sync, athena_benchmark
├── airflow/dags/          # master_batch, hourly_zone_agg, daily_payout
├── ml/                    # features.py, train.py, score.py
├── genai/                 # tools.py, assistant_app.py
├── dashboards/            # Power BI .pbix
├── tests/                 # unit tests, drills/, reconciliation.py
└── docs/
```

14. Deliverables

- Source code repository with the structure above and a README describing setup and run steps.
- Docker Compose environment for all open-source components.

- Working streaming pipeline producing surge multipliers every 60 seconds, stored in HBase and Delta.
- Hive star schema, S3/Glue/Athena tables and a Databricks DLT Medallion pipeline.
- Three Airflow DAGs with an automated daily email report.
- Power BI dashboard file (.pbix) with heat map, utilisation gauge and revenue/km trends.
- Trained demand forecast model with evaluation report (R^2 , MAE, RMSE) and MLflow run.
- GenAI Ops assistant with its tool definitions.
- Test report covering the three failure drills, reconciliation and KPI results.
- Final project report and demo presentation.

15. Risks and Mitigations

Risk	Impact	Mitigation
Laptop cannot run every service at once	Slow or crashing stack	Run in profiles (ingestion, streaming, batch); single-node HDFS/HBase; limit Spark memory
Apache Sqoop is retired (moved to the Apache Attic in 2021)	Setup issues on new Hadoop versions	Use a pinned Sqoop 1.4.7 + Hadoop image; keep a Spark JDBC import script as backup
Cloud free-tier limits	Unexpected cost	Small data samples, S3 lifecycle rules, AWS budget alert, Athena workgroup scan limit
DLT or connector features differ on the free Databricks tier	Pipeline cannot run as designed	Verify early in week 1; fall back to plain Delta streaming jobs with the same checks
Forecast R^2 below 0.70	Model SLA missed	Add lag and weather features, tune Random Forest, aggregate sparse zones
External API downtime	Gaps in live feed	Retry with backoff; replay producer keeps the stream going

16. Conclusion

This project delivers a complete, working data platform for dynamic surge pricing: batch and streaming ingestion, a real-time surge engine refreshed every 60 seconds, a warehouse and lakehouse for history, scheduled pipelines, dashboards, a demand forecast and a GenAI assistant. It uses every tool named in the problem statement, implemented in Python, and it will be judged against clear, measurable targets and three failure drills that prove the pipeline is reliable.